

GROUP WORK PROJECT # _1_
Finance
GROUP NUMBER: ____15919____

MScFE 632: Machine Learning in

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Aman	India	amanbhoot84@gmail.com	
Makhale Mogoboya	South Africa	mogoboyamakhale@gmail.com	
David Olutunde Daniel	USA	daviddaniels48@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1	Aman
Team member 2	Makhale
Team member 3	David Olutunde Daniel

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

A Marketing Handbook of Machine Learning Techniques for Finance

MScFE 632 — Machine Learning in Finance

Group Work Project #1

Submission Information

Team name	Team 15919
Course	MScFE 632 — Machine Learning in Finance
Deliverable	Group Work Project #1 (written article, no code)
Group size	3 members

Member	Categories	Focus
Team A: Aman Aman	1 (LASSO), 4 (Trees)	Regression and trees
Team B: Makhale Mogoboya	2 (k-means), 5 (LDA)	Clustering and discriminant
Team C: David Olutunde Daniel	3 (PCA), 6 (SVM), 7 (NN)	Factors, kernels, networks

All code and its output live in the separate Jupyter notebook and its PDF export. This document holds only the written article, formulas, and figures.

1 LASSO Regression

Basics

LASSO is a linear regression with a rule that keeps it honest. A plain regression fits a line by making the squared errors as small as it can. LASSO adds a penalty on the total size of the coefficients, and that penalty has a useful side effect: it pushes the weakest coefficients all the way to zero. So the model fits a line and picks features in one step. It sits in the family of penalized linear models next to Ridge and Elastic Net.

Equations

LASSO minimizes the squared error plus an L1 penalty:

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j|.$$

The first term rewards fit. The second, scaled by λ , punishes big coefficients. Set λ to zero and you are back to ordinary regression; raise it and more coefficients snap to exactly zero. That absolute-value shape is what creates the zeros, and it is the one thing that separates LASSO from Ridge.

Advantages and disadvantages

The upside is that LASSO cleans house. It selects variables for you, fights overfitting on noisy return data, and leaves a short model that is easy to explain. It shines when you believe only a handful of your many signals really matter. The downside shows up when two signals are highly correlated, since it tends to keep one and drop the other almost by coin flip. It also shrinks the coefficients it keeps, so their sizes are biased, and it needs the features scaled first or the penalty treats them unfairly.

Features and guide

LASSO handles wide feature sets, gives sparse output, assumes a roughly linear link, and does not deal with missing values on its own. It takes a matrix of standardized numeric features (here lagged returns, momentum, a moving-average gap, volatilities, volume change, and two peer signals) and a continuous target, next-day return. It returns a set of coefficients, most of them zero, a chosen penalty, and predictions.

Hyperparameters

The one that matters is λ (called **alpha** in scikit-learn). We picked it by five-fold cross-validation over a grid from 10^{-5} to 1, and the winner was about 0.0006. The rest, like the iteration cap and the intercept, are minor.

Illustration

Figure 1 traces each coefficient as the penalty grows from left to right. One by one the lines fall to zero; the dashed line is the penalty we settled on.

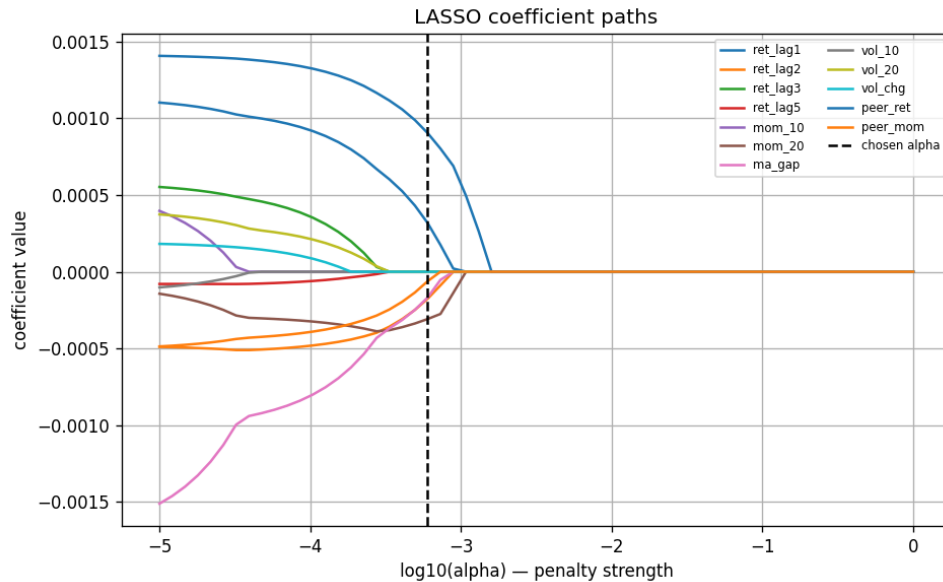


Figure 1: LASSO coefficient paths. As the penalty rises, weak signals drop to zero. The dashed line marks the cross-validated choice.

At our penalty, six signals survived and six went to zero. The strongest was the peer return, then the asset's own one-day lagged return, then 20-day momentum. On the test set the RMSE was about 0.018 and the R^2 sat near 0.003. That tiny R^2 is the honest truth of next-day return prediction: almost none of tomorrow's move is written in yesterday's numbers.

Tags: LASSO, L1 regularization, feature selection, sparse regression, shrinkage, return prediction

2 k-means Clustering

Basics

k-means is unsupervised, so there is nothing to predict. You choose how many groups you want, k , and it sorts the points so that members of a group sit close together. It works by distance to a set of cluster centers. Here we cluster the eight assets by their behavior: average return, volatility, market beta, and average correlation to the basket.

Equations

k-means minimizes the total within-group squared distance:

$$\arg \min_{\{C_1, \dots, C_k\}} \sum_{m=1}^k \sum_{x \in C_m} \|x - \mu_m\|^2, \quad \mu_m = \frac{1}{|C_m|} \sum_{x \in C_m} x.$$

Here μ_m is the center of group m . The algorithm loops two steps until nothing moves: assign each point to its nearest center, then recompute each center as the mean of its members.

Advantages and disadvantages

It is simple, fast, and scales well, and the groups it gives are clean and easy to act on, which helps for finding peer baskets or market regimes without any labels. The catch is that you must pick k yourself, and a bad choice gives misleading groups. It assumes clusters are roughly round

and similar in size, so odd shapes trip it up. It also depends on the random starting centers, which is why we use several restarts, and it is sensitive to outliers and scaling.

Features and guide

k-means is distance-based, needs standardized inputs, wants k up front, and does not handle missing values. It takes one row of numeric features per item (here one row per asset) and a value of k , and returns a group label for each item plus the center of each group.

Hyperparameters

The key knob is k . We scanned $k = 2$ through 6 and let the silhouette score and the elbow of the inertia curve decide. The silhouette peaked at $k = 2$ (about 0.55), so we used two clusters. The number of restarts and the initialization method are the secondary settings.

Illustration

Figure 2 shows how we chose k , and Figure 3 shows the result.

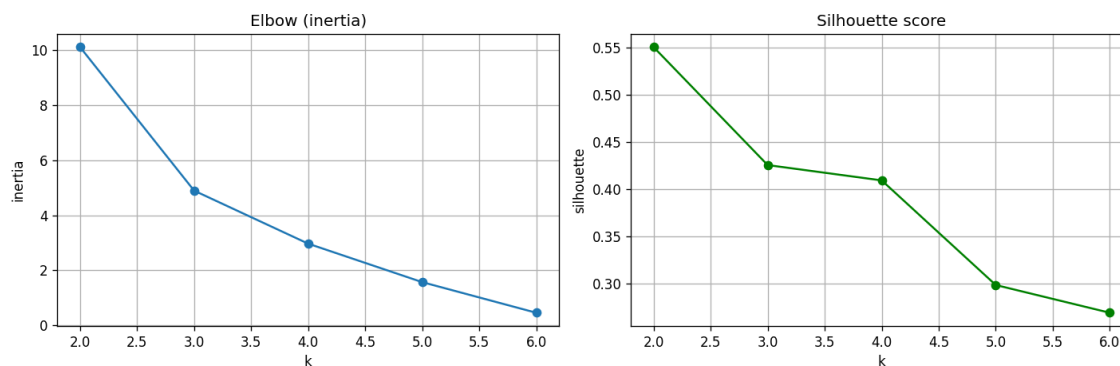


Figure 2: Choosing k . Left: inertia falls as k grows, and we look for the elbow. Right: the silhouette peaks at $k = 2$.

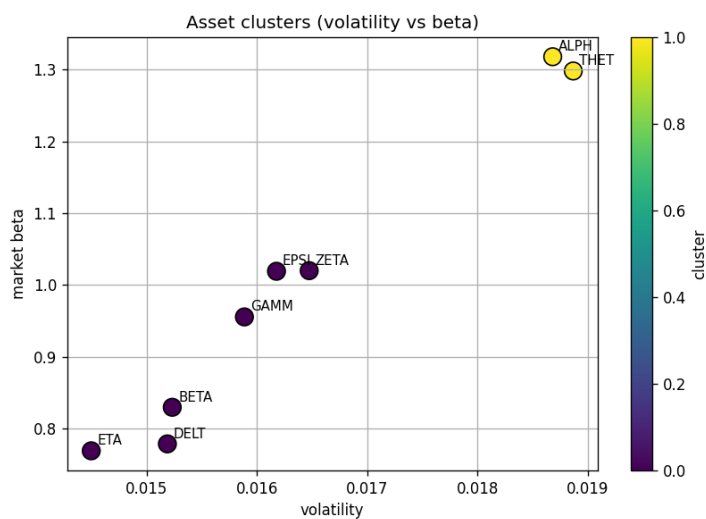


Figure 3: The two clusters on volatility versus beta. Two jumpy, high-beta names break away from the calmer pack.

The split was clean. A small group of high-volatility, high-beta names (ALPH and THET) stood apart from the calmer, lower-beta majority. A manager could use that to avoid loading up on the risky pair.

Tags: k-means, clustering, unsupervised learning, centroids, silhouette, peer grouping, portfolio construction

3 Principal Component Analysis

Basics

PCA shrinks many correlated variables into a few new ones, called principal components, that do not overlap with each other and are ranked by how much variation they explain. With stock returns the first component usually behaves like the market itself.

Equations

PCA finds the eigenvectors of the covariance matrix Σ of the data:

$$\Sigma v_k = \lambda_k v_k, \quad \Sigma = \frac{1}{n-1} X^\top X.$$

Each direction v_k is a component, and its eigenvalue λ_k says how much variance runs along it. The share explained by component k is $\lambda_k / \sum_j \lambda_j$, and you keep the top few that cover most of the variance.

Advantages and disadvantages

PCA cuts a pile of correlated signals down to a few independent factors, which trims noise and speeds up whatever model comes next, and those factors often line up with real forces like the market or a sector tilt. It is also a clean way to see high-dimensional data in two dimensions. The trouble is that the new components are blends of the originals, so they can be hard to name. It only catches linear structure, it is very sensitive to scaling, and dropping small components can quietly toss out something useful.

Features and guide

PCA is linear, unsupervised, and scale-sensitive, and it produces uncorrelated outputs. It takes a standardized numeric matrix (here the daily returns of the eight assets) and returns the component directions, the variance each explains, and the data projected onto the top components.

Hyperparameters

There is little to tune. The main call is how many components to keep, usually driven by a target like 90% of the variance or the elbow of the scree plot. Whether to standardize the inputs is the other real decision.

Illustration

Figure 4 shows the scree plot and how each asset loads on the first two components.

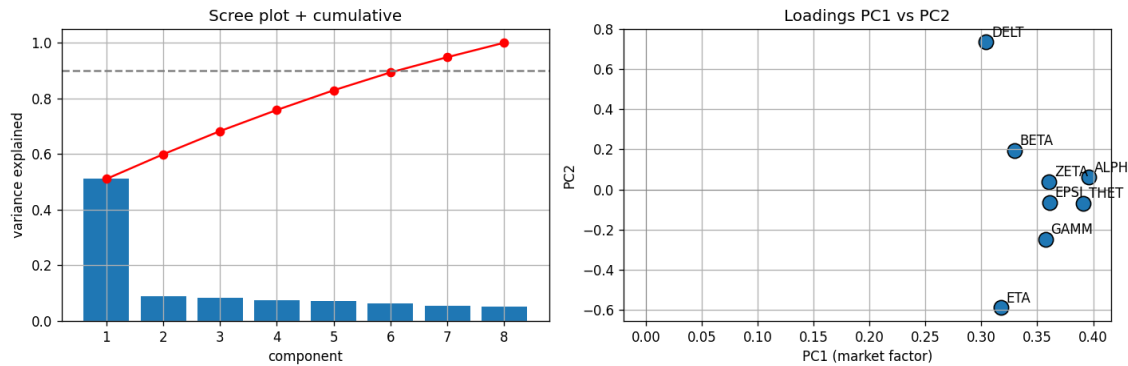


Figure 4: Left: variance explained per component with the running total. Right: asset loadings on PC1 and PC2. Every asset loads the same sign on PC1, the market factor.

The first component alone explained about 51% of all the movement across the eight assets, and every asset loaded on it with the same sign. That is the fingerprint of a market factor: when PC1 moves, everything moves with it. The other components each explained under 10% and captured the smaller quirks that make one name differ from another.

Tags: PCA, principal components, dimensionality reduction, eigenvectors, factor model, market factor

4 Classification Trees

Basics

A classification tree predicts a category by asking a chain of yes/no questions about the features. Each question splits the data, and the splits continue until you reach a leaf that hands back a class. It carves the feature space into rectangles. Here it predicts whether the focus asset closes up tomorrow.

Equations

A tree picks each split to make the resulting groups as pure as it can. For a node with class shares p_c , the two common purity measures are Gini and entropy:

$$\text{Gini} = 1 - \sum_c p_c^2, \quad \text{Entropy} = - \sum_c p_c \log_2 p_c.$$

At each step the tree chooses the feature and cutoff that cut impurity the most, and it repeats until a stopping rule kicks in.

Advantages and disadvantages

Trees are easy to read, since the path from top to leaf is just a set of rules, and they catch non-linear effects and feature interactions without you spelling them out. They need almost no preprocessing, do not care about scaling, and rank feature importance for free. The flip side is that a single deep tree overfits fast and is unstable, so a small change in the data reshuffles the splits. Its cuts are axis-aligned, so a diagonal boundary comes out blocky, and on noisy return data one tree rarely beats the baseline by much.

Features and guide

A tree is non-linear, readable, and scale-free, and with the right build it tolerates some missing values. It takes numeric or categorical features and a categorical target, up versus down, and

returns a class per row, class probabilities, and an importance ranking.

Hyperparameters

We tuned the maximum depth, the minimum samples per leaf, and the split rule (Gini or entropy) with five-fold cross-validation. The best mix was Gini, a depth of five, and at least twenty samples per leaf. Those limits are exactly what stop the tree from memorizing noise.

Illustration

Figure 5 shows the top of the fitted tree.

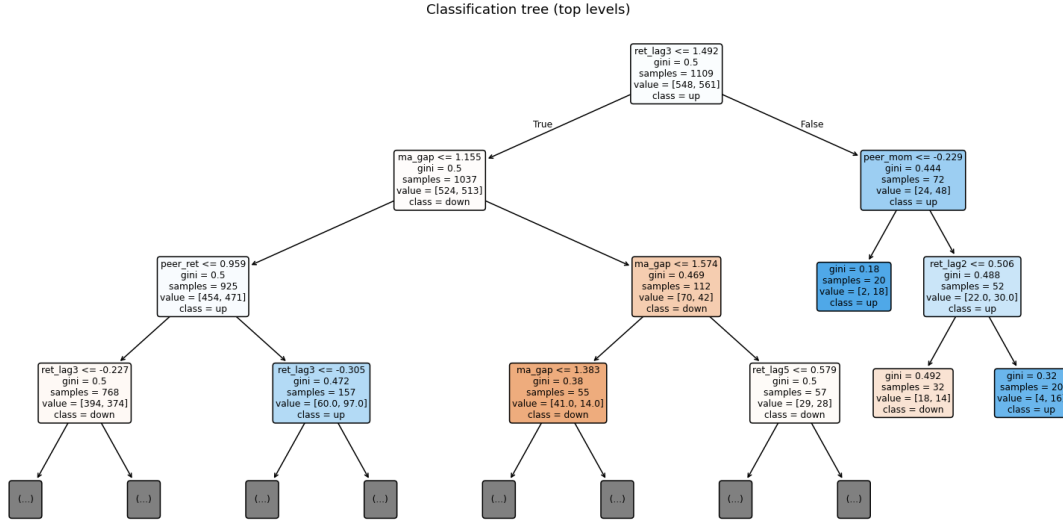


Figure 5: The top levels of the classification tree. Each box shows the rule, the impurity, how many samples reach it, and the predicted class.

The tree leaned hard on the three-day lagged return, which carried about half the total importance, followed by the moving-average gap. On the test set it reached 0.532 accuracy, a whisker above the 0.508 baseline. That thin edge is normal: a lone tree pulls only a little signal out of daily returns.

Tags: decision tree, classification, Gini impurity, entropy, information gain, feature importance

5 Linear Discriminant Analysis

Basics

LDA assumes each class comes from a bell-shaped cloud and that all classes share the same spread, then finds the straight line that best splits them. It is fast, has almost nothing to tune, and also gives a low-dimensional view that pulls the classes apart.

Equations

LDA sends a point x to the class with the highest score:

$$\delta_c(x) = x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c,$$

where μ_c is the class mean, Σ is the shared covariance, and π_c is the prior share of class c . Because the covariance is shared, the parts that depend on x are linear, which is why the boundary is a straight line.

Advantages and disadvantages

LDA is quick and nearly tuning-free, and it works well when the classes really are roughly bell-shaped with a similar spread. It doubles as a way to reduce dimensions while keeping the classes apart, and it stays stable on small samples because it leans on strong, simple assumptions. The weakness is those same assumptions: financial returns have fat tails, so the bell-shape rarely holds, and the boundary is only ever a straight line. It is also thrown off by outliers, which distort the means and the covariance.

Features and guide

LDA is linear and generative, assumption-heavy, and close to parameter-free. It takes numeric features and a categorical target and returns class predictions, class probabilities, and a projection onto discriminant axes.

Hyperparameters

There is barely anything to set. The real choices are the solver and whether to shrink the covariance estimate, which helps when features outnumber samples. We used the default SVD solver with no shrinkage.

Illustration

Figure 6 shows the one-dimensional projection LDA learned.

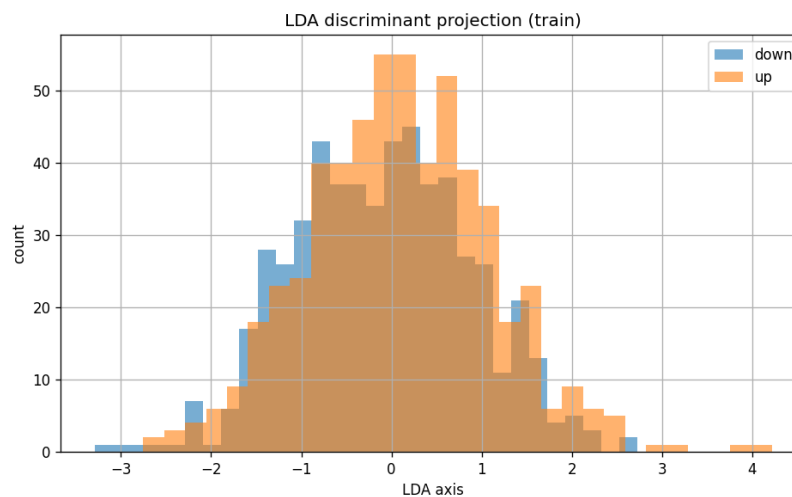


Figure 6: The single LDA axis with up and down days overlaid. The heavy overlap shows how little separation there is in daily return signals.

The two histograms sit almost on top of each other, which tells the story straight: on this data the up and down days barely separate along a straight line. Test accuracy came in at 0.478, just under the baseline. LDA is a fine, fast tool, but the signal here simply is not linear enough for it to grab.

Tags: LDA, linear discriminant analysis, Gaussian classifier, dimensionality reduction, generative model

6 Support Vector Machine

Basics

An SVM finds the boundary that separates two classes with the widest possible gap between them. Only the points nearest the boundary, the support vectors, actually shape it. With the kernel trick it can bend that boundary into complex shapes without building new features by hand.

Equations

The soft-margin SVM solves

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The term $\|w\|$ sets the margin width, the slack variables ξ_i let a few points fall on the wrong side, and C trades margin against errors. The map ϕ enters through a kernel, most often the RBF kernel $K(x, x') = \exp(-\gamma \|x - x'\|^2)$.

Advantages and disadvantages

An SVM handles non-linear boundaries cleanly through kernels, works well in high dimensions, and generalizes nicely when the classes are well separated and the settings are right. The costs are real, though: it is slow to train as the sample grows, it lives and dies by the tuning of C and γ , it does not give probabilities without extra work, and once a non-linear kernel is in play it is hard to interpret.

Features and guide

An SVM is non-linear with kernels, margin-based, scale-sensitive, and heavy on memory for big data. It takes standardized numeric features and a categorical target and returns class predictions, the set of support vectors, and, with calibration, probabilities.

Hyperparameters

The big two are C , how much to punish misclassified points, and γ , how wiggly the RBF boundary gets, plus the choice of kernel. We grid-searched with five-fold cross-validation and landed on an RBF kernel with $C = 1$ and $\gamma = 1$. That fit used 1,109 support vectors, a strong hint that the classes overlap heavily.

Illustration

Figure 7 draws the boundary on the two most informative features.

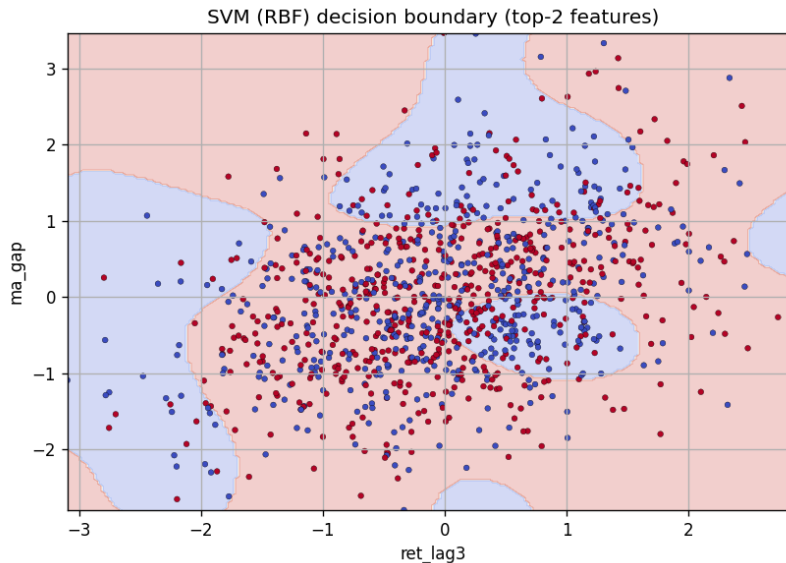


Figure 7: The RBF-SVM boundary on the two strongest features. The colored regions are the model's up and down zones; the heavy mixing of points shows why accuracy stays near baseline.

The boundary curves, which is the kernel doing its job, but the two classes are so tangled that the curve cannot separate them cleanly. Test accuracy was 0.489. The large count of support vectors confirms that almost every point sits right on the fuzzy border.

Tags: SVM, support vector machine, kernel trick, RBF kernel, margin, support vectors

7 Neural Networks

Basics

A neural network stacks simple units in layers. Each unit takes a weighted sum of its inputs, runs it through a non-linear function, and passes it on. Stack enough layers and the network can learn complicated, non-linear patterns. Here we use a small multilayer perceptron to predict up versus down days.

Equations

For a layer with weights W , bias b , input a , and activation f , the output is

$$a^{(\ell+1)} = f(W^{(\ell)}a^{(\ell)} + b^{(\ell)}), \quad f_{\text{ReLU}}(z) = \max(0, z).$$

Training minimizes a loss (here cross-entropy) by gradient descent, using backpropagation to work out how each weight should shift:

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W},$$

where η is the learning rate.

Advantages and disadvantages

Given enough data, a network can approximate very complex, non-linear relationships, its shape can grow or shrink to fit the problem, and it learns useful feature combinations on its own. The costs are that it is hungry for data and overfits easily on small, noisy financial samples, it is a black

box that is hard to explain to a risk committee, it carries many settings and longer training times, and it is sensitive to scaling and to the random seed.

Features and guide

A network is highly non-linear, flexible, data-hungry, and scale-sensitive. It takes standardized numeric features and a target and returns predictions, class probabilities, and the learned weights.

Hyperparameters

We tuned the hidden-layer sizes, the L2 strength **alpha**, and the activation with cross-validation. The best model had two hidden layers of 32 and 16 units, ReLU activation, and **alpha** of 0.01, and we used early stopping to halt before it overfit. The learning rate, batch size, and number of epochs are the other important knobs in general.

Illustration

Figure 8 shows the training loss falling as the network learns.

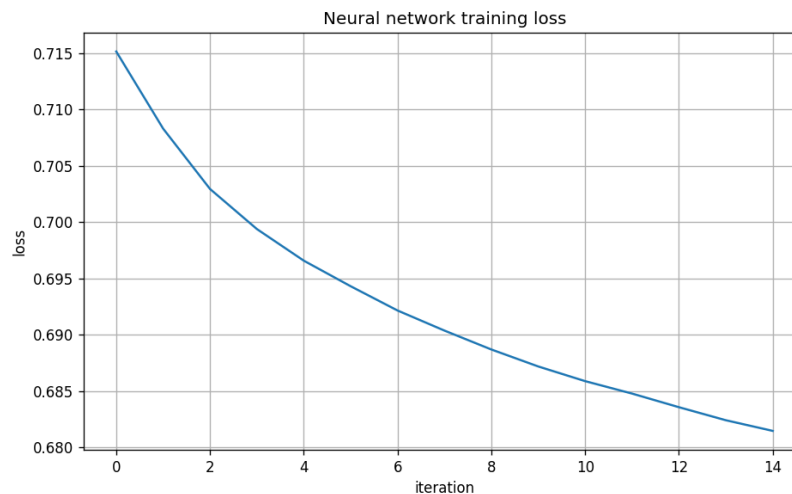


Figure 8: Training loss against iteration. It drops fast, then flattens, which is where early stopping steps in.

The curve behaves as it should: a quick early drop, then a plateau. Test accuracy landed at 0.505, essentially the baseline. On richer data with more features and more history, the extra flexibility would have room to help; on daily direction alone, it has little to hold onto.

Tags: neural network, MLP, deep learning, backpropagation, ReLU, activation function

8 How Hyperparameters Are Tuned

Every model here has settings that are not learned from the data on their own. These are the hyperparameters, and choosing them well is most of the craft. Our recipe was the same across models: lay out a grid of candidate values, split the training data into folds, train on some folds and score on the held-out one, rotate, and keep the setting with the best average score. Because the data is a time series, we kept the final test window strictly after the training window, so nothing from the future leaked back.

The main tools were grid search, which tries every combination on a grid, and cross-validation, which averages the score over several folds so one lucky split cannot fool us. We used four or five folds. For the unsupervised methods, which have no target, we used internal scores instead: the silhouette for k-means and the explained-variance curve for PCA. The table below lists what we actually tuned and what we chose.

Model	Tuned	Chosen
LASSO	penalty λ	≈ 0.0006 (5-fold CV)
k-means	clusters k	$k = 2$ (silhouette 0.55)
PCA	components kept	7 for 90% variance; PC1 $\approx 51\%$
Classification tree	depth, min leaf, criterion	depth 5, min leaf 20, Gini
LDA	solver, shrinkage	SVD, no shrinkage
SVM	C , γ , kernel	$C = 1$, $\gamma = 1$, RBF
Neural network	layers, alpha, activation	(32,16), alpha 0.01, ReLU

The lesson for the desk is that tuning is not decoration. The tree only beat the baseline because we capped its depth; left unlimited, it would have memorized noise. The SVM needed the right γ or it would have drawn a bubble around every single point. Good tuning is the line between a model that generalizes and one that flatters you in backtest and fails live.

9 Marketing Alpha

Here is the case for why these tools belong on the desk, built from what each one does best.

Machine learning does not replace judgment; it sharpens it. Each tool answers a different question a manager actually asks. When the question is which of my many signals matter, LASSO returns a short, honest list and drops the rest, so we stop paying attention to noise. When the question is which of my names move together, k-means draws the peer groups, and in our test it cleanly split two high-risk names off from the calmer pack, exactly the concentration a manager wants to see before sizing anything. When the question is how many real forces drive the whole basket, PCA answered that a single market factor explains about half of all the movement, which is the first thing any risk model needs to know.

On the prediction side, the four classifiers each bring something. Trees give rules a person can read and defend. LDA is instant and needs almost no tuning, so it is a great first look. SVMs bend the boundary when the relationship is truly non-linear. Networks scale up to richer data where the simpler models flatten out.

We are also straight about the limits, because that is what earns trust. On daily direction alone, every classifier landed near the coin-flip line. That is not a failure of the tools; it is the market saying that tomorrow's direction is close to unpredictable from yesterday's numbers alone. The real edge comes from feeding these same tools better inputs: longer history, fundamentals, alternative data, and cross-asset context. The methods are proven. The edge is in the data and the discipline we bring to them, and that combination is what makes Team Alpha worth attaching to a revenue stream.

10 Comparing Models

The table sums up how each model handles a set of practical features a desk cares about. Entries are short qualitative calls.

Feature	LASSO	k-means	PCA	Tree	LDA	SVM	NN
Missing data	Poorly	Poorly	Poorly	Well	Poorly	Poorly	Poorly
Non-linearity	Poorly	N/A	Poorly	Well	Poorly	Well	Well
Interpretable	High	High	Medium	High	Medium	Low	Low
Needs scaling	Yes	Yes	Yes	No	Yes	Yes	Yes
Speed (big data)	Fast	Fast	Fast	Fast	Fast	Slow	Medium
Feature selection	Yes	No	Reduces	Implicit	No	No	Implicit
Supervised	Yes	No	No	Yes	Yes	Yes	Yes
Overfit risk	Low	Medium	Low	High	Low	Medium	High
Probabilities	N/A	No	No	Yes	Yes	Calib.	Yes

Read across and no single model wins everywhere. Trees are the most forgiving about messy inputs. LASSO and PCA are the cleanest for cutting noise. SVMs and networks buy non-linear power at the cost of speed and readability. The right pick depends on the question and on how much you need to explain the answer.

11 For the Investment Manager

This part is for decision-makers, so it skips the model names and jargon.

We tested several data-driven tools on a basket of eight stocks to see whether we could call tomorrow's direction and to understand how the names relate. The direction call was, honestly, close to a coin flip. That is a useful result on its own: do not bet the book on short-term direction signals built only from recent price moves.

The grouping and factor work was far more useful. It showed that a single broad force drives about half of the whole basket's movement, and that two of the eight names carry noticeably more risk and swing much harder than the rest. Both of those are things you can act on right now.

So the recommended course is simple. First, treat short-horizon direction forecasts as weak tilts at most, never standalone strategies. Second, watch concentration: the two jumpy names should not be sized as if they behave like the calm majority, because they will amplify a bad day. Third, since one common factor dominates the basket, hedging that single broad exposure removes most of the risk more efficiently than hedging name by name. The dominant driver is broad market movement, which hits every position at once; the secondary one is the split between the calm majority and the volatile pair. Because daily direction is so weakly predictable, timing bets should carry little weight, while the structural calls on concentration and factor exposure carry the real leverage.

References

- Altman, Edward I. “Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy.” *The Journal of Finance*, vol. 23, no. 4, 1968, pp. 589–609.
- Avellaneda, Marco, and Jeong-Hyun Lee. “Statistical Arbitrage in the U.S. Equities Market.” *Quantitative Finance*, vol. 10, no. 7, 2010, pp. 761–782.
- Chinco, Alex, Adam D. Clark-Joseph, and Mao Ye. “Sparse Signals in the Cross-Section of Returns.” *The Journal of Finance*, vol. 74, no. 1, 2019, pp. 449–492.
- Huang, Wei, Yoshiteru Nakamori, and Shou-Yang Wang. “Forecasting Stock Market Movement Direction with Support Vector Machine.” *Computers & Operations Research*, vol. 32, no. 10, 2005, pp. 2513–2522.
- Krauss, Christopher, Xuan Anh Do, and Nicolas Huck. “Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the S&P 500.” *European Journal of Operational Research*, vol. 259, no. 2, 2017, pp. 689–702.
- Nanda, S. R., B. Mahanty, and M. K. Tiwari. “Clustering Indian Stock Market Data for Portfolio Management.” *Expert Systems with Applications*, vol. 37, no. 12, 2010, pp. 8793–8798.
- Sorensen, Eric H., Keith L. Miller, and Chee K. Ooi. “The Decision Tree Approach to Stock Selection.” *The Journal of Portfolio Management*, vol. 27, no. 1, 2000, pp. 42–52.
- Supporting texts.* James, Gareth, et al. *An Introduction to Statistical Learning with Applications in Python*. Springer, 2023. Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. 2nd ed., Springer, 2009.